Lab 3 Report:

MNIST Classification with FCN

Nate Chaoweeraprasit

In [106…

```
# Import necessary packages

%matplotlib inline

import matplotlib.pyplot as plt

import torch
import torchvision
import numpy as np
```

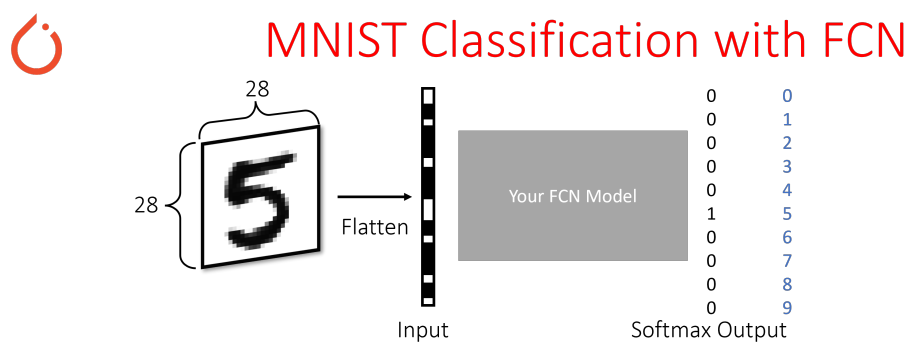
In [106…

```
from IPython.display import Image # For displaying images in colab jup
```

In [106…

```
Image('lab3_exercise.PNG', width = 1000)
```

Out[106…



In this exercise, you will classify handwritten digits (28 x 28) using your own **Fully Connected Network Architecture.**

Prior to training your neural net, 1) Flatten each digit into 1D array of size 784, 2) Normalize the dataset using standard scaler and 3) Split the dataset into train/validation/test.

Design your own neural net architecture with your choice of hidden layers, activation functions, optimization method etc.

Your goal is to **achieve a testing accuracy of >90%**, with no restrictions on epochs.

Demonstrate the performance of your model via plotting the **training loss, validation accuracy** and printing out the **testing accuracy.**

Plot the testing samples where your model failed to classify correctly and print your model's best guess for each of them 44

Prepare Data

In [106…

```
# Load MNIST Dataset in Numpy

# 1000 training samples where each sample feature is a greyscale image
# 1000 training targets where each target is an integer indicating the
```

```
mnist_train_features = np.load('mnist_train_features.npy')
mnist_train_targets = np.load('mnist_train_targets.npy')

# 100 testing samples + targets
mnist_test_features = np.load('mnist_test_features.npy')
mnist_test_targets = np.load('mnist_test_targets.npy')

# Print the dimensions of training sample features/targets
print(mnist_train_features.shape, mnist_train_targets.shape)
# Print the dimensions of testing sample features/targets
print(mnist_test_features.shape, mnist_test_targets.shape)
```

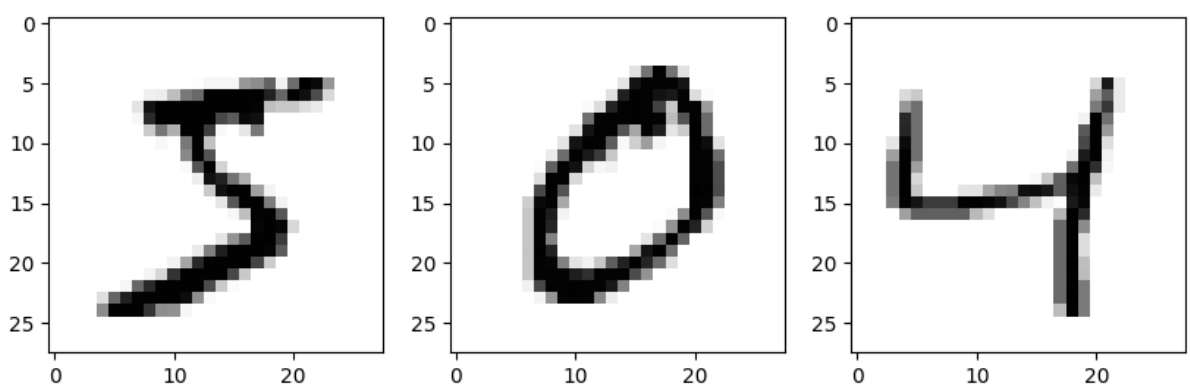
```
(1000, 28, 28) (1000,)
(100, 28, 28) (100,)
```

In [106…

```
# Let's visualize some training samples

plt.figure(figsize = (10, 10))

plt.subplot(1,3,1)
plt.imshow(mnist_train_features[0], cmap = 'Greys')

plt.subplot(1,3,2)
plt.imshow(mnist_train_features[1], cmap = 'Greys')

plt.subplot(1,3,3)
plt.imshow(mnist_train_features[2], cmap = 'Greys')
```

Out[106…

```
<matplotlib.image.AxesImage at 0x144ea1760>
```



In [106…

```
# Reshape features via flattening the images
# The reshaped training/testing features should have shape (1000, 784)
# np.reshape() function could be useful here

mnist_train_features = mnist_train_features.reshape(1000, -1)
mnist_test_features = mnist_test_features.reshape(100, -1)
```

In [106…

```
# scaling function
def scale_data(arr):

    # exception gate; ensure array is 2D array
```

```
    if arr.ndim != 2:
        raise ValueError("array isn't 2D")

    # create copy so we don't affect original array
    scaled_data = arr.copy()

    # iterate by column
    for i in range(scaled_data.shape[1]):

        # for each column, find mean and stdev
        column_mean = np.mean(scaled_data[:, i])
        column_stdev = np.std(scaled_data[:, i])

        # then do (x-mean)/std to find scaled value at each point
        scaled_data[:, i] = (scaled_data[:, i] - column_mean) / column

    return scaled_data
```

In [106…

```
# Scale the dataset according to standard scaling
# mnist_train_features = scale_data(mnist_train_features)
# mnist_test_features = scale_data(mnist_test_features)

mnist_train_features = mnist_train_features / 255
mnist_test_features = mnist_test_features / 255
```

In [106…

```
# Split dataset into Train (900 samples), Validation (100 samples), Te
train_num = 900

mnist_val_features = mnist_train_features[train_num:]
mnist_val_targets = mnist_train_targets[train_num:]

mnist_train_features = mnist_train_features[:train_num]
mnist_train_targets = mnist_train_targets[:train_num]

print(mnist_train_features.shape)
print(mnist_val_features.shape)
print(mnist_test_features.shape)
```

```
(900, 784)
(100, 784)
(100, 784)
```

Define Model

In [ ]:

```
class mnistClassification(torch.nn.Module):

    def __init__(self, input_dim, hidden_dim, output_dim): # Feel free

        super(mnistClassification, self).__init__()

        # two fully connected layers + relu activation + dropout layer
```

```
        self.fc1 = torch.nn.Linear(input_dim, hidden_dim)
        self.relu = torch.nn.ReLU()
        self.fc2 = torch.nn.Linear(hidden_dim, output_dim)
        self.dropout = torch.nn.Dropout(p = 0.25)

    def forward(self, x):

        x = self.fc1(x)
        x = self.relu(x)
        x = self.dropout(x)
        x = self.fc2(x)
        return x
```

Define Hyperparameters

In [ ]:
```
# Initialize our neural network model with input and output dimensions
model = mnistClassification(input_dim = 784,
                            hidden_dim = 256,
                            output_dim = 10)

# Define the learning rate and epoch
learning_rate = 0.0045
epochs = 25

# Define loss function and optimizer
loss_func = torch.nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr = learning_rate)

# Run this line if you have PyTorch GPU version
if torch.cuda.is_available():
    model.cuda()

model
```

Out[ ]:
```
mnistClassification(
  (fc1): Linear(in_features=784, out_features=256, bias=True)
  (relu): ReLU()
  (fc2): Linear(in_features=256, out_features=10, bias=True)
  (dropout): Dropout(p=0.25, inplace=False)
)
```

Identify Tracked Values

In [107…
```
# Placeholders for training loss and validation accuracy during traini
# Training loss should be tracked for each iteration (1 iteration -> s
# Validation accuracy should be evaluated every 'Epoch' (1 epoch -> fu
# If using batch gradient, 1 iteration = 1 epoch

train_loss_list = np.zeros((epochs,))
val_loss_list = np.zeros((epochs,))
```

```
val_accuracy_list = np.zeros((epochs,))
```

Train Model

In [107…

```
import tqdm

# Convert the training, validation, testing dataset (NumPy arrays) int

train_inputs = torch.from_numpy(mnist_train_features).float()
train_targets = torch.from_numpy(mnist_train_targets).long()

val_inputs = torch.from_numpy(mnist_val_features).float()
val_targets = torch.from_numpy(mnist_val_targets).long()

test_inputs = torch.from_numpy(mnist_test_features).float()
test_targets = torch.from_numpy(mnist_test_targets).long()

# Training Loop ------------------------------------------------------

# include batch gradient descent if you want (optional)

for epoch in tqdm.trange(epochs):

    # forward pass to compute outputs and loss
    train_outputs = model(train_inputs)
    train_loss = loss_func(train_outputs, train_targets)

    # backpropagation to compute gradients
    optimizer.zero_grad()
    train_loss.backward()

    # update weights
    optimizer.step()

    # track loss + accuracy
    train_loss_list[epoch] = train_loss.item()
    val_loss_list[epoch] = loss_func(model(val_inputs), val_targets).i

    # Compute Validation Accuracy ------------------------------------

    with torch.no_grad():

        val_outputs = model(val_inputs)
        prediction = torch.argmax(val_outputs, dim=1)
        correct = (prediction == val_targets).type(torch.FloatTensor)

        val_accuracy_list[epoch] = correct.mean()
```

```
100%|██████████| 25/25 [00:00<00:00, 79.98it/s]
```

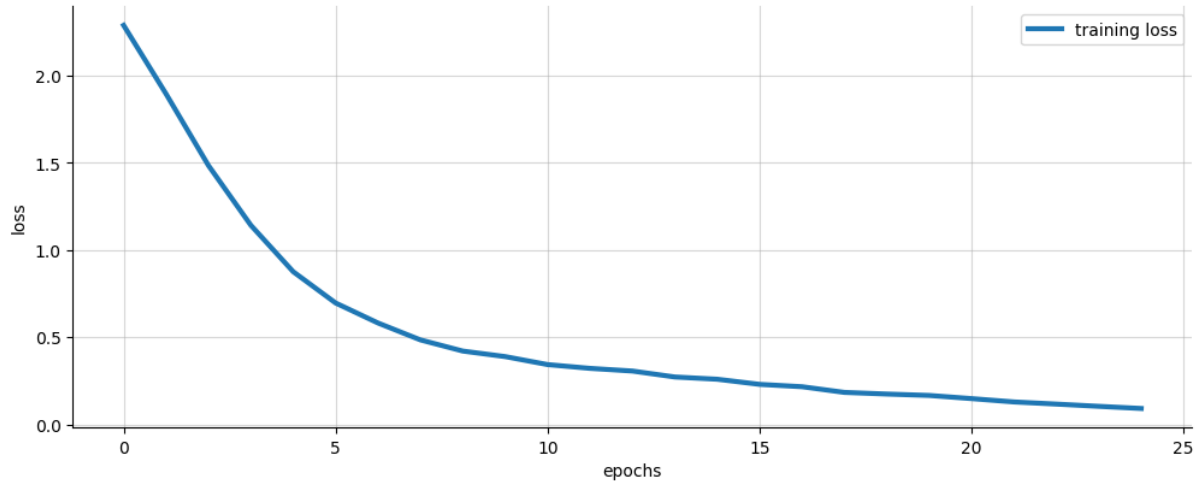
Visualize and Evaluate Model

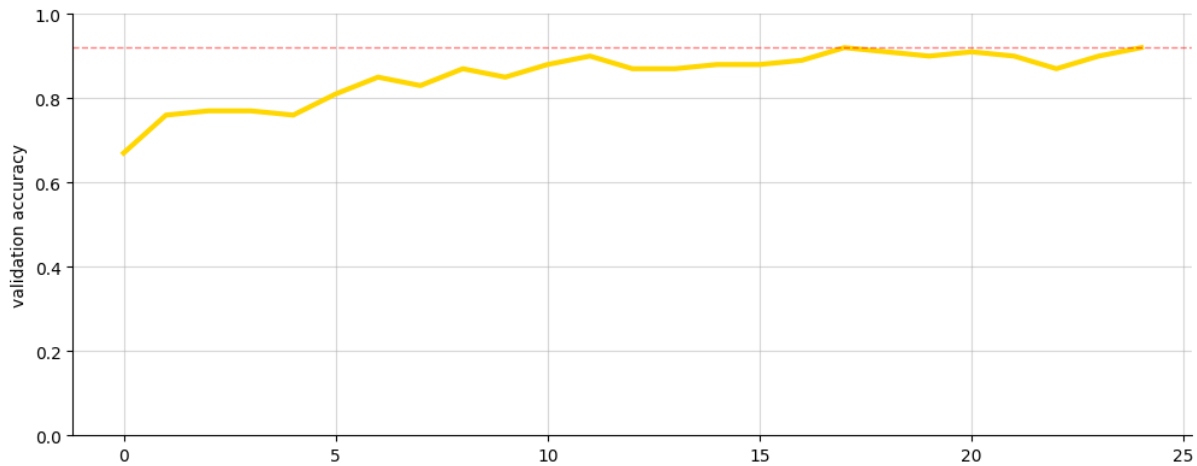
In [107…

```
# Import seaborn for prettier plots

import seaborn as sns
```

In [107…

```
# Visualize training loss

plt.figure(figsize = (12,10))

# Visualize training loss with respect to iterations (1 iteration -> s
plt.subplot(2, 1, 1)
plt.plot(train_loss_list, linewidth = 3, label = 'training loss')
plt.ylabel("loss")
plt.xlabel("epochs")
plt.legend()
plt.grid(alpha = 0.5)
sns.despine()

# Visualize validation accuracy with respect to epochs
plt.subplot(2, 1, 2)
plt.plot(val_accuracy_list, linewidth = 3, color = 'gold')
plt.axhline(max(val_accuracy_list), linewidth = 1, linestyle = '--', c


plt.ylim(0,1)
plt.ylabel("validation accuracy")
plt.grid(alpha = 0.5)
sns.despine()
```



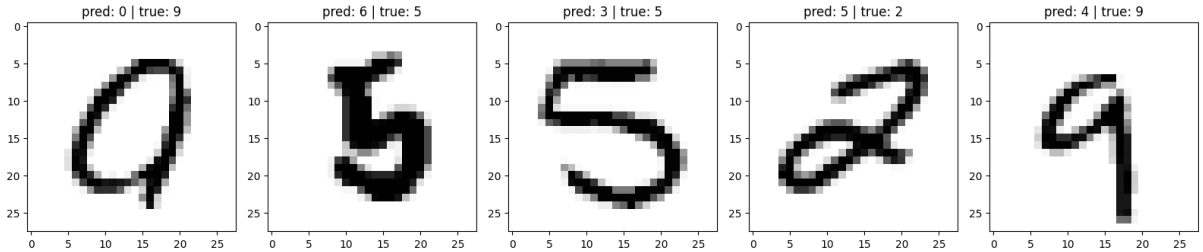


In [107…

```
# Compute the testing accuracy

with torch.no_grad():

    test_outputs = model(test_inputs)
    prediction = torch.argmax(test_outputs, dim=1)

    # compare predictions with targets and find accuracy
    correct = (prediction == test_targets).type(torch.FloatTensor)

print(f"testing accuracy: {correct.mean():.4f}")
```

```
testing accuracy: 0.9100
```

In [108…

```
# Plot 5 incorrectly classified testing samples and print the model pr
# You can use np.reshape() to convert flattened 1D array (768,) -> (28

with torch.no_grad():
    val_outputs = model(val_inputs)
    prediction = torch.argmax(val_outputs, dim=1)

# fetch index of incorrect classifications
incorrect_index = (prediction != val_targets).nonzero(as_tuple=True)[0

plt.figure(figsize = (16, 10))
```

```
for i in range(5):
    plt.subplot(1, 5, i+1)
    plt.imshow(val_inputs[incorrect_index[i]].reshape(28, 28), cmap =
    plt.title(f"pred: {prediction[incorrect_index[i]].item()} | true:

plt.tight_layout()
plt.show()
```